Pre-training of Language Models

Learning signals, training choices, and a modern recipe

Lê Văn Thức - Bảo Quý Định Tân - Lê Phạm Thành Nhân

NLP Nâng Cao

Roadmap: six questions about pre-training

01 Pretraining Introduction

02 Pretraining Motivation

03 Objectives and architectures

04 Scaling

05 Qwen3: a modern case study

How do the choices come together?

06 Extensions and boundaries

What transfers beyond text, and what comes next?

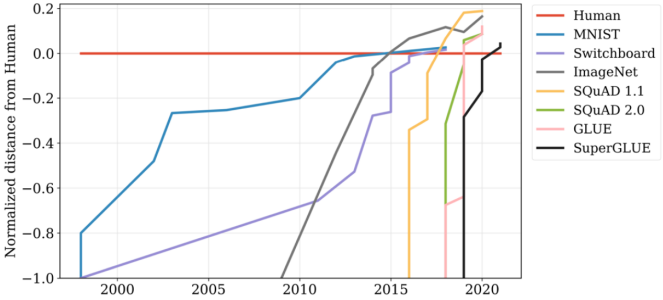
PART 1 / 06

Pretraining Introduction

We'll cover: Why we use Pretraining

Six steps from learning signals to modern pre-training

The pretraining revolution



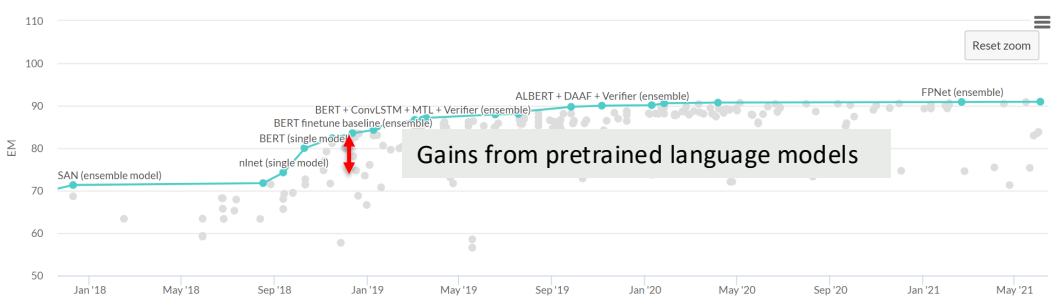
Source: CS224N Lecture 7: Pretraining

A shift in NLP

- Rapid benchmark improvements accompanied pretrained models.
- A general-purpose model can transfer across tasks.

Pretraining became a foundation for modern NLP systems.

A closer look: NLP leaderboard gains



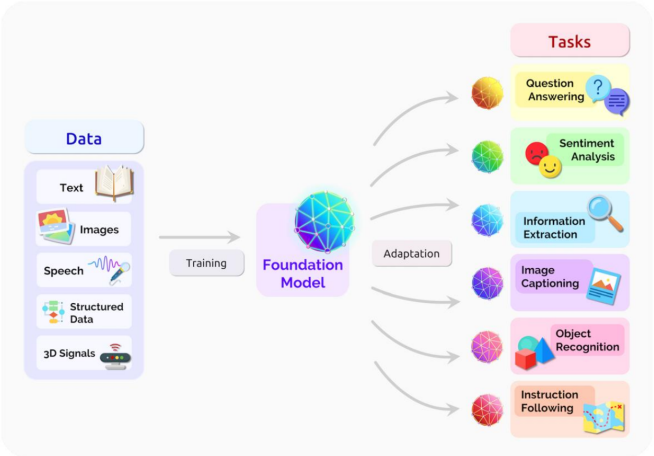
Source: CS224N Lecture 7: Pretraining

Horizontal axis: time (2018–2021)

Vertical axis: exact-match score

Gains associated with pretrained LMs.

Scaling self-supervised learning on internet data



Large + diverse
Handle broad datasets.
No manual labels
Targets come from data.
Compute-aware
Scale within a budget.

Source: CS224N Lecture 7: Pretraining

Learn general structure once; reuse the resulting parameters for downstream tasks.

PART 2 / 06

Pretraining Motivation

We'll cover: Pretraining motivation and how?

Six steps from learning signals to modern pre-training

A word's meaning depends on its context

Distributional semantics (Firth, 1957)

You shall know a word by the company it keeps.

The missing piece (Firth, 1935)

...the complete meaning of a word is always contextual...

I **record** the **record**.

verb noun

The same word form can have different meanings depending on its sentence.

Source: CS224N Lecture 7: Pretraining

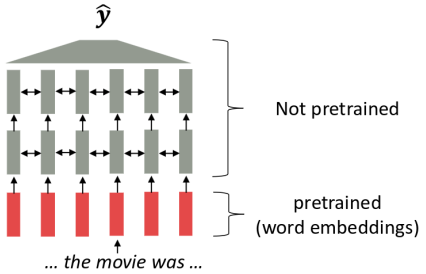
Before model pretraining: static word embeddings

The 2017-era recipe

1. Start with pretrained word embeddings (no context!)
2. Learn how to incorporate context in an LSTM or Transformer while training on the task.

Issues:

1. The training data we have for our downstream task (like question answering) must be sufficient to teach all contextual aspects of language.
2. Most of the parameters in our network are randomly initialized!



[Recall, *movie* gets the same word embedding, no matter what sentence it shows up in]

Source: CS224N Lecture 7: Pretraining

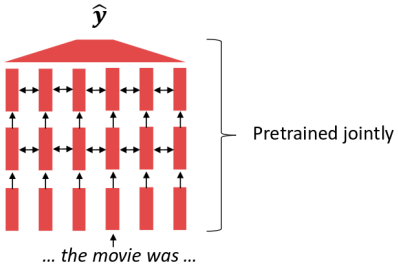
Now: pretrain the whole contextual model

The modern recipe:

1. All (or almost all) parameters in NLP networks are initialized via pretraining.
2. Pretraining methods hide parts of the input from the model, and train the model to reconstruct those parts.

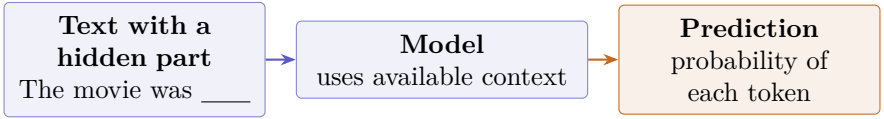
Exceptionally effective at building strong:

1. representations of language.
2. parameter initializations for strong NLP models.
3. probability distributions over language that we can sample from.



[This model has learned how to represent entire sentences through pretraining]

Why is reconstructing missing text useful?



Where is supervision?
The original text supplies the answer.

What gets learned?
Whatever regularities help predict it.

Reconstructing text: 1/6 – factual knowledge

Stanford University is located in _____, California.

Possible answer

Stanford

Useful signal

Recall a location from text.

Correct completion can depend on knowledge found across documents.

Source: CS224N Lecture 7: Pretraining

Reconstructing text: 2/6 – grammar

I put _____ fork down on the table.

Possible answer

the / a / my

Useful signal

Use syntax and article / determiner choices.

Several completions may be grammatical; the objective scores their likelihood.

Source: CS224N Lecture 7: Pretraining

Reconstructing text: 3/6 – reference

The woman crossed the street,
checking traffic over _____ shoulder.

Possible answer

her

Useful signal

Use information about the subject earlier in the sentence.

Predicting a pronoun requires resolving nearby context.

Source: CS224N Lecture 7: Pretraining

Reconstructing text: 4/6 – categories

I went to the ocean to see
fish, turtles, seals, and _____.

Possible answer

dolphins / whales

Useful signal

Use semantic relationships among listed items.

Word prediction can reward knowledge of category membership.

Source: CS224N Lecture 7: Pretraining

Reconstructing text: 5/6 – sentiment

Two hours watching it were worth no more
than the popcorn and drink.
The movie was _____.

Possible answer

bad / disappointing

Useful signal

Integrate sentiment across multiple sentences.

The best continuation depends on the attitude of the whole passage.

Source: CS224N Lecture 7: Pretraining

Reconstructing text: 6/6 – patterns

The sequence goes
1, 1, 2, 3, 5, 8, 13, 21, _____.

Possible answer

34

Useful signal

Recognize a numerical sequence (Fibonacci).

Pattern completion can reward structured regularities, not just local grammar.

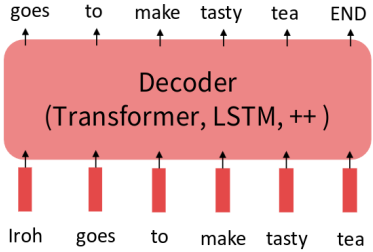
Source: CS224N Lecture 7: Pretraining

Pretraining through language modeling

Language modeling

$$p_\theta(w_t \mid w_{1:t-1})$$

Predict a word from preceding context. Large text corpora supply many training examples.



Pretraining

1. Train a neural language model on lots of text.
2. Save its parameters as a checkpoint.

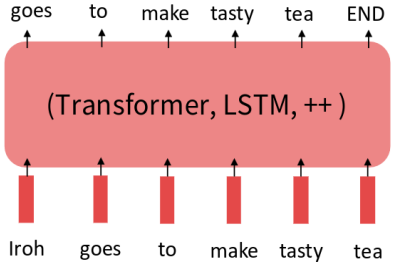
Pretraining converts abundant unlabeled text into reusable model parameters.

Source: CS224N Lecture 7: Pretraining

The pretraining / fine-tuning paradigm

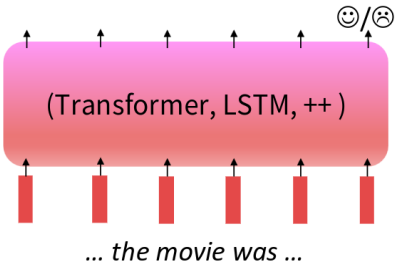
Step 1: Pretrain (on language modeling)

Lots of text; learn general things!



Step 2: Finetune (on your task)

Not many labels; adapt to the task!



Transfer learned parameters first; adapt to a specific task second.

Source: CS224N Lecture 7: Pretraining

Where does pretraining text come from?

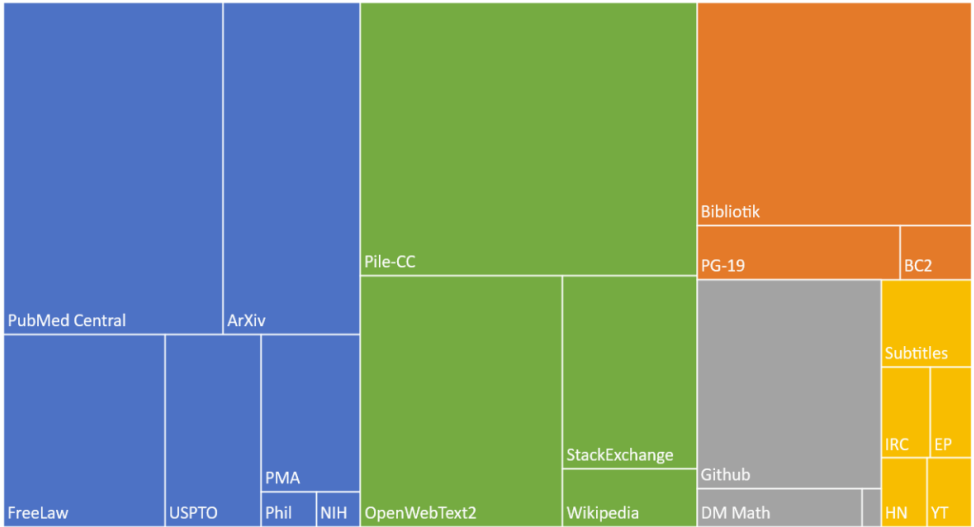
CS224N compares the publicly described training sources of earlier models.

Model	Reported training data
BERT	BookCorpus, English Wikipedia
GPT-1	BookCorpus
GPT-3	Common Crawl, WebText, English Wikipedia, two book datasets (“Books 1” and “Books 2”)
GPT-3.5+	Undisclosed

Data is not one homogeneous source: mixtures of web, books and curated text shape exposure.

Source: CS224N Lecture 7: Pretraining

The Pile: a multi-source text collection

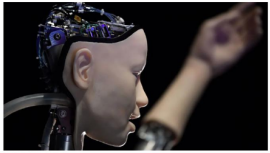


Sometimes it also memorizes copyrighted material

AI Art Generators Spark Multiple Copyright Lawsuits

Getty and a trio of artists sued AI art generators in separate suits accusing the companies of copyright infringement for pilfering their works.

BY WINSTON CHO | JANUARY 17, 2023 4:05PM





Anthropic fires back at music publishers' AI copyright lawsuit

By Blake Brittain

January 17, 2024 3:30 PM PST · Updated 19 days ago



ARTICLE

Insights from the Pending Copilot Class Action Lawsuit

October 4, 2023

Bloomberg Law

By Daniel R. Mello, Jr.; Jenevieve J. Maerker; Matthew C. Berntsen; Ming-Tao Yang

GitHub Inc. offers a cloud-based platform that is popular among many software programmers for hosting and sharing source code, and collaborating on source code drafting. GitHub's artificial

The Times Sues OpenAI and Microsoft Over A.I. Use of Copyrighted Work

Millions of articles from The New York Times were used to train chatbots that now compete with it, the lawsuit said.

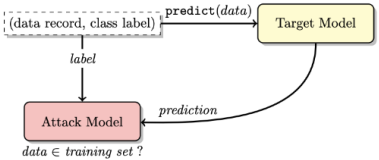
Share free access · 1.3K

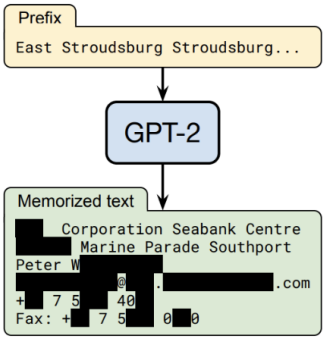


Sometimes it learns some things we don't want...

- Membership inference lets you recover parts of the training data.
- Sometimes this training data is semi-private material from the web (addresses, emails).

Source: CS224N Lecture 7: Pretraining





PART 03 / 06

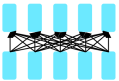
Objectives and architectures

We'll cover:

Six steps from learning signals to modern pre-training

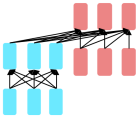
Pretraining for three types of architectures

The neural architecture influences the type of pretraining, and natural use cases.



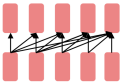
Encoders

- Gets bidirectional context – can condition on future!
- How do we train them to build strong representations?



Encoder-Decoders

- Good parts of decoders and encoders?
- What's the best way to pretrain them?

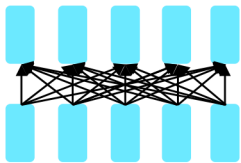


Decoders

- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words

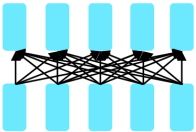
Part I: Encoder Pretraining

Masked Language Modeling and BERT



Pretraining for three types of architectures

The neural architecture influences the type of pretraining, and natural use cases.



Encoders

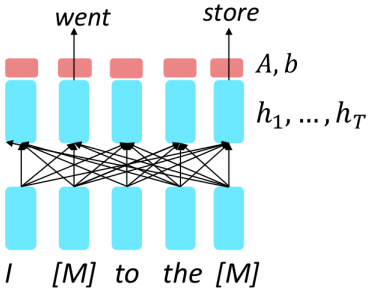
- Gets bidirectional context – can condition on future!
- How do we train them to build strong representations?

Pretraining encoders: what objective to use?

Encoders have bidirectional context, so ordinary next-token language modeling is not appropriate with unmasked inputs.

Masked Language Modeling (MLM)

- Replace some input tokens by `[MASK]`.
- Predict the original token at masked positions.
- Compute loss only at selected positions.



$$\mathbf{h}_{1:T} = \text{Encoder}(\widetilde{\mathbf{x}})$$

$$\mathcal{L}_{\text{MLM}} = -\sum_{i \in M} \log p_\theta(x_i \mid \widetilde{\mathbf{x}})$$

Notation:

- $\mathbf{x}$: Original token sequence.
- $\widetilde{\mathbf{x}}$: Corrupted input sequence.

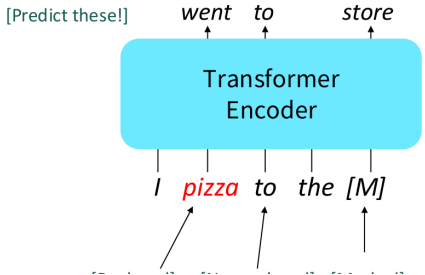
BERT: Bidirectional Encoder Representations from Transformers

Devlin et al. (2018) released pretrained Transformer encoders trained with Masked LM.

BERT's 15% selected-token procedure

- **80%**: replace selected token with `[MASK]`.
- **10%**: replace with a random token.
- **10%**: leave unchanged, but predict it.

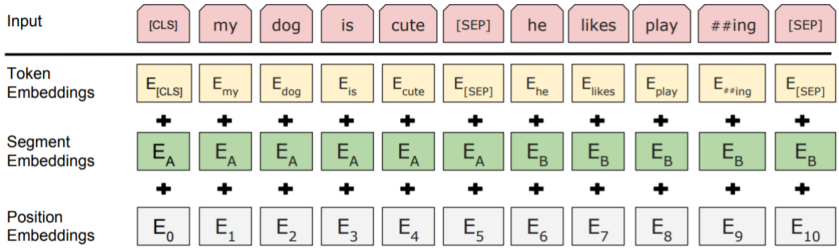
This reduces reliance on seeing `[MASK]` at training time, because fine-tuning inputs normally contain no mask tokens.



Source: Devlin et al., 2018

BERT: input representation and NSP

BERT pretraining can feed two contiguous text chunks as one sequence, combining **token**, **segment**, and **position** embeddings.



- Original BERT also trained *next sentence prediction* (NSP): does the second segment follow the first, or is it randomly sampled?
- Later work argued that NSP is not necessary.

Source: Devlin et al., 2018; Liu et al., 2019

Extensions of BERT: more data and compute

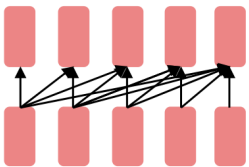
A key observation from RoBERTa: **more data and more compute** can improve encoder pretraining without changing the basic Transformer architecture.

Model	data	bsz	steps	SQuAD (v1.1/2.0)	MNLI-m	SST-2
RoBERTa						
with BOOKS + WIKI	16GB	8K	100K	93.6/87.3	89.0	95.3
+ additional data (§3.2)	160GB	8K	100K	94.0/87.7	89.3	95.6
+ pretrain longer	160GB	8K	300K	94.4/88.7	90.0	96.1
+ pretrain even longer	160GB	8K	500K	**94.6/89.4**	**90.2**	**96.4**
$\text{BERT}_{\text{LARGE}}$						
with BOOKS + WIKI	13GB	256	1M	90.9/81.8	86.6	93.7

Source: Liu et al., 2019

Part II: Decoder Pretraining

Causal Language Modeling and GPT



Pretraining for three types of architectures: decoders

Focus: decoder-only Transformer

A causal attention mask yields **left-to-right next-token modeling**.

- Natural to sample tokens sequentially from the model.
- Training objective: $\mathcal{L} = -\sum_{t=1}^{T} \log p_\theta(w_t \mid w_{<t})$.

Pretraining decoders

When using language model pretrained decoders, we can ignore that they were trained to model $p_\theta(w_t \mid w_{1:t-1})$.

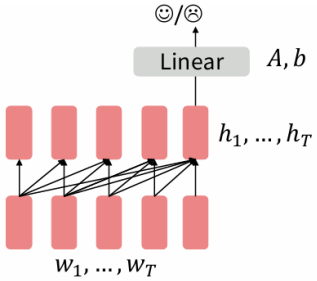
We can finetune them by training a classifier on the last word's hidden state.

$$h_{1:T} = \text{Decoder}(w_{1:T})$$

$$\hat{\mathbf{y}} = \text{softmax}(Ah_T + b)$$

Where A and b are randomly initialized and specified by the downstream task.

Gradients backpropagate through the whole network.



Note how the linear layer hasn't been pretrained and must be learned from scratch.

Pretraining decoders

It's natural to pretrain decoders as language models and then use them as generators, finetuning their $p_\theta(w_t \mid w_{1:t-1})$!

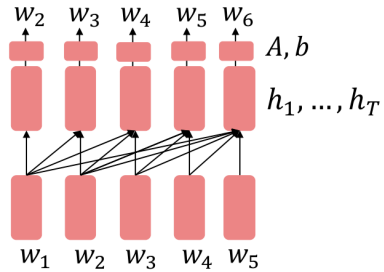
This is helpful in tasks where the output is a **sequence** with a vocabulary like that at pretraining time!

- Dialogue (context = dialogue history)
- Summarization (context = document)

$$h_{1:T} = \text{Decoder}(w_{1:T})$$

$$p_\theta(w_t \mid w_{1:t-1}) = \text{softmax}(Ah_{t-1} + b)$$

Where A, b were pretrained in the language model!



Note how the linear layer has been pretrained.

GPT: Generative pretrained transformer

2018's GPT was a big success in pretraining a decoder!!

Method	MNLI-m	MNLI-mm	SNLI	SciTail	QNLI	RTE
ESIM + ELMo [44] (5x)	-	-	89.3	-	-	-
CAFE [58] (5x)	80.2	79.0	89.3	-	-	-
Stochastic Answer Network [35] (3x)	80.6	80.1	-	-	-	-
CAFE [58]	78.7	77.9	88.5	83.3		
GenSen [64]	71.4	71.3	-	-	82.3	59.2
Multi-task BiLSTM + Attn [64]	72.2	72.1	-	-	82.1	**61.7**
Finetuned Transformer LM (ours)	**82.1**	**81.4**	**89.9**	**88.3**	**88.1**	56.0

Source: Radford et al., 2018

Increasingly convincing generations: GPT-2

GPT-2 scales up the original GPT to **1.5 billion parameters** and trains on more data. Its language-model samples showed improved natural-language fluency.

Context (human-written): In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

GPT-2: The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science.

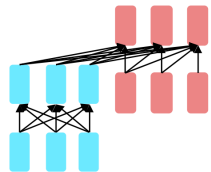
Now, after almost two centuries, the mystery of what sparked this odd phenomenon is finally solved.

Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans. Pérez noticed that the valley had what appeared to be a natural fountain, surrounded by two peaks of rock and silver snow.

Source: GPT-2 examples reproduced in CS224N source lecture

Part III: Encoder–Decoder Pretraining

Prefix LM, denoising, and T5



Pretraining for three types of architectures: encoder–decoders

Focus: Encoder–Decoder

The encoder can build a bidirectional representation of the input; the decoder can then generate a target sequence autoregressively.

- Can pretraining combine **representation learning** and **generation**?
- What should the encoder see?
- What should the decoder reconstruct?

Pretraining encoder–decoders: prefix language modeling

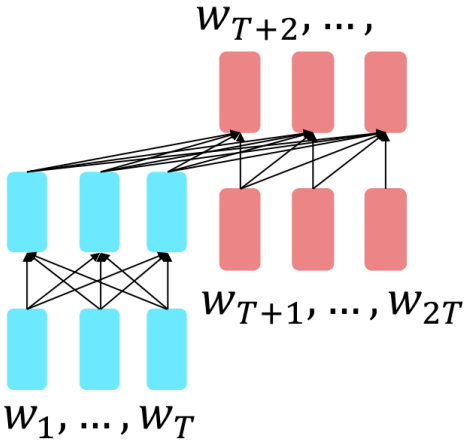
For encoder–decoders, we could do something like language modeling, but where a prefix of every input is provided to the encoder and is not predicted.

$$\mathbf{h}_{1:T} = \text{Encoder}(w_{1:T})$$

$$\mathbf{h}_{T+1:2T} = \text{Decoder}(w_{T+1:2T}, \mathbf{h}_{1:T})$$

$$\hat{w}_i \sim A\mathbf{h}_i + b, \quad i > T$$

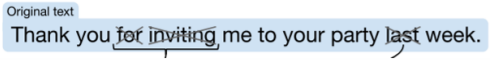
The encoder portion benefits from bidirectional context; the decoder portion is used to train the whole model through language modeling.



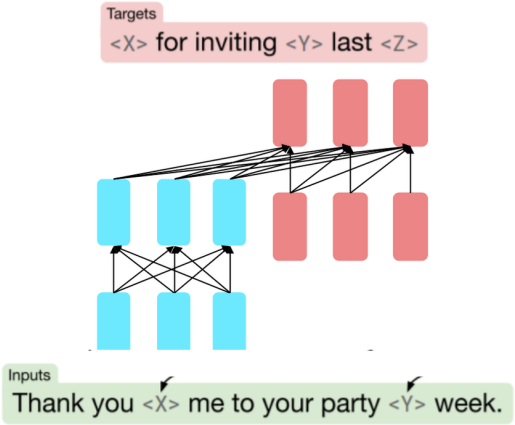
Pretraining encoder-decoders: what pretraining objective to use?

What Raffel et al., 2018 found to work best was **span corruption**. Their model: **T5**.

Replace different-length spans from the input with unique placeholders; decode out the spans that were removed!



This is implemented in text preprocessing: it's still an objective that looks like **language modeling** at the decoder side.



Source: Raffel et al., 2018

T5: comparing architectures and objectives

In the cited experimental setting, encoder–decoders with denoising outperform the corresponding language-modeling configurations on the aggregate metrics examined.

Architecture	Objective	Params	Cost	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ Encoder-decoder	Denoising	$2P$	M	**83.28**	**19.24**	**80.88**	**71.36**	**26.98**	**39.82**	**27.65**
Enc-dec, shared	Denoising	P	M	82.81	18.78	**80.63**	**70.73**	26.72	39.03	**27.46**
Enc-dec, 6 layers	Denoising	P	$M/2$	80.88	18.97	77.59	68.42	26.38	38.40	26.95
Language model	Denoising	P	M	74.70	17.93	61.14	55.02	25.09	35.28	25.86
Prefix LM	Denoising	P	M	81.82	18.61	78.94	68.11	26.43	37.98	27.39
Encoder-decoder	LM	$2P$	M	79.56	18.59	76.02	64.29	26.27	39.17	26.86
Enc-dec, shared	LM	P	M	79.60	18.13	76.35	63.50	26.62	39.17	27.05
Enc-dec, 6 layers	LM	P	$M/2$	78.67	18.26	75.32	64.06	26.13	38.42	26.89
Language model	LM	P	M	73.78	17.54	53.81	56.51	25.23	34.31	25.38
Prefix LM	LM	P	M	79.68	17.84	76.87	64.86	26.28	37.51	26.76

Source: Raffel et al., T5; results depend on the tasks and setup shown

PART 04 / 06

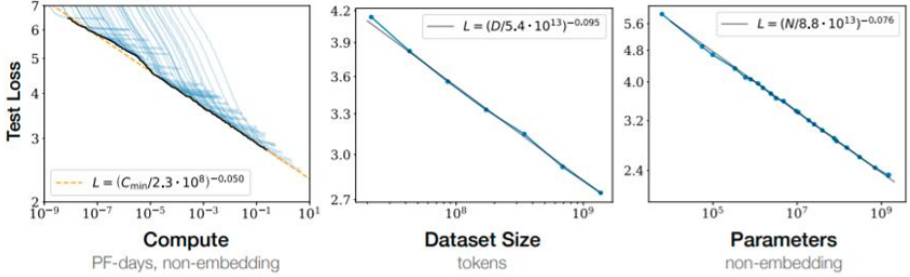
Scaling

We'll cover:

Six steps from learning signals to modern pre-training

Why scale? Scaling laws

Empirical observation: scaling up models leads to reliable gains in perplexity.



Source: Scaling plots from CS224N

A fixed compute budget

For a dense Transformer, a useful training approximation is:

$$C \approx 6ND$$

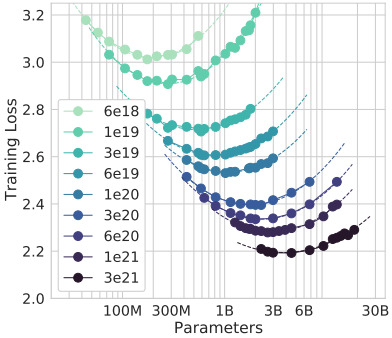
C	Training FLOPs
N	Number of parameters
D	Processed training tokens

At fixed C, doubling N approximately halves D. Capacity and training exposure compete for the budget.

This dense-model approximation omits important attention and systems costs.

Source: Hoffmann et al. (2022), §3.3 and Appendix F

Compute-optimal training



Source: Hoffmann et al. (2022), Figure 3, left panel

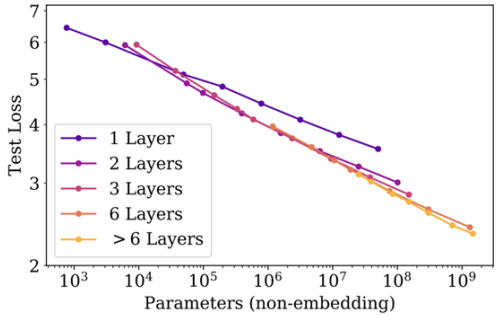
Each curve uses a fixed FLOP budget.

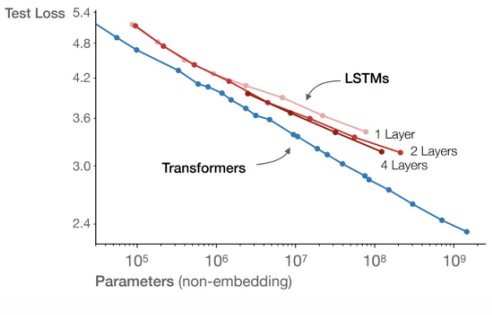
- Too small: limited capacity.
- Too large: too few tokens.
- The valley estimates a better allocation.

$$(N^*, D^*) = \arg \min_{6ND=C} L(N, D)$$

Scaling laws for architectural decisions

Predictable scaling helps us make intelligent decisions about architectures etc.





Source: Plots from CS224N lecture, p. 51

Scaling Laws: From Experiments to Optimal Pretraining

Key idea: Scaling laws are empirical, not predefined rules. We learn them from experiments to guide pretraining decisions.

1. Experiment
Train smaller models with different N, D, and architectures.

$\Longrightarrow$

2. Model Scaling
Fit scaling laws to predict loss at larger scales.

$\Longrightarrow$

3. Optimize
Choose model size, data, and architecture under compute budget.

$$(N^*, D^*) = \arg\min_{N,D} L(N, D) \quad \text{s.t.} \quad C(N, D) \leq C_{\max}$$

Takeaway

Scaling efficiency: using training compute effectively

GPT-3 had **175B parameters** and was trained on approximately **300B tokens**.

Did OpenAI strike the right parameter-token data to get the best model? No.

Model	Size (# Parameters)	Training Tokens
LaMDA (Thoppilan et al., 2022)	137 Billion	168 Billion
GPT-3 (Brown et al., 2020)	175 Billion	300 Billion
Jurassic (Lieber et al., 2021)	178 Billion	300 Billion
Gopher (Rae et al., 2021)	280 Billion	300 Billion
MT-NLG 530B (Smith et al., 2022)	530 Billion	270 Billion
Chinchilla	70 Billion	1.4 Trillion

Conclusion: This 70B parameter model is better than the much larger other models!

Source: Chinchilla / compute-optimal scaling comparison in CS224N lecture, p. 52

Chinchilla: an empirical comparison

Models trained with similar total training compute:

Model	Parameters	Training tokens	MMLU 5-shot
Gopher	280B	300B	60.0%
Chinchilla	**70B**	**1.4T**	**67.6%**

In this study, a smaller model trained on more tokens performed better.

Training-compute optimal ≠ deployment optimal

One training run

Minimize validation loss at a fixed training FLOP budget:

$$\min_{6ND=C} L(N, D).$$

A deployed model

Meet a quality target while considering training and repeated inference:

$$\text{cost}_{\text{life}} = \text{cost}_{\text{train}} + Q\,\text{cost}_{\text{query}}.$$

Llama 3 provides a concrete example

Meta estimated about 200B tokens as Chinchilla-optimal for an 8B model, yet reported continued gains through 15T tokens. A smaller model trained longer can be attractive when inference is frequent.

Source: Meta (2024), Introducing Llama 3, "Scaling up pretraining"

PART 5 / 06

Pre-training in Practice: Qwen3

What does pre-training a modern LLM look like in practice?

We'll cover: model architecture, data, staged training, and base-model evidence

Six steps from learning signals to modern pre-training

From Theory to Practice

We know the learning objective. What does a real pre-training recipe require?

$$\mathcal{L}_{\text{CLM}}(\theta) = -\sum_{t} \log p_\theta(x_t \mid x_{<t})$$

The causal next-token objective remains the foundation.

The engineering choices

Architecture and scale

Corpus quality and data mixture

Training stages and context length

Evaluation of the base checkpoint

Qwen3 is a concrete case study of how these choices fit together—without changing the underlying next-token task.

Source: Qwen Team (2025), Sections 2–3

Inside Qwen3

One family, two architectures

Type	Qwen3 model sizes
Dense	0.6B, 1.7B, 4B
	8B, 14B, 32B
MoE	30B-A3B
	235B-A22B

Shared Transformer foundations: GQA, RoPE, RMSNorm, SwiGLU, QK-Norm.

Example: Qwen3-235B-A22B

Total parameters	**235B**
Active per token	**22B**
Experts	128
Selected per token	8

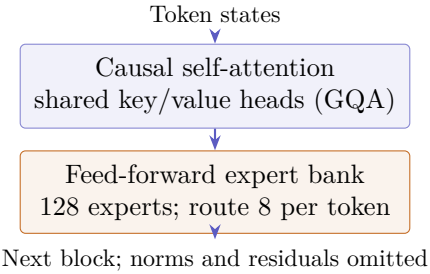
"A22B" refers to the approximate activated parameter count, not an extra model.

Dense models use all their FFN weights per token; MoE models route each token through a subset of expert FFNs.

Source: Qwen Team (2025), Section 2, Tables 1–2

Mixture-of-Experts

How can a 235B-parameter model activate only about 22B per token?



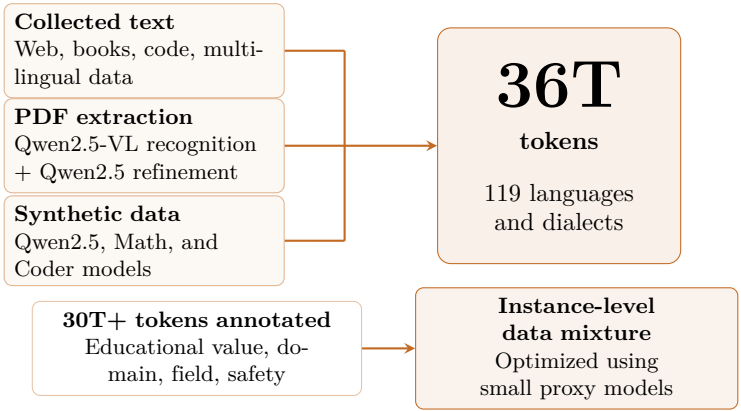
Stored capacity
All experts contribute to the total 235B weights.

Per-token computation
Shared layers plus 8 selected experts contribute to roughly 22B active weights.

MoE separates total model capacity from the parameters used for one token; 22B is not the size of the entire checkpoint.

Building the Training Dataset

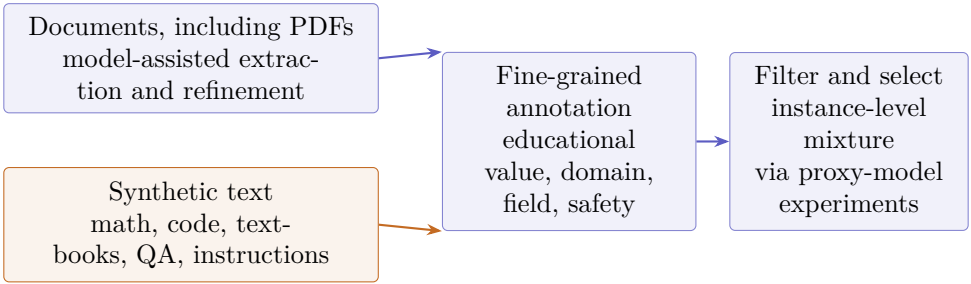
Where do Qwen3's 36 trillion pre-training tokens come from?



Qwen3 scales pre-training through data extraction, synthetic generation, and fine-grained mixture optimization.

Beyond Raw Data

Why not simply collect more text and train uniformly on it?



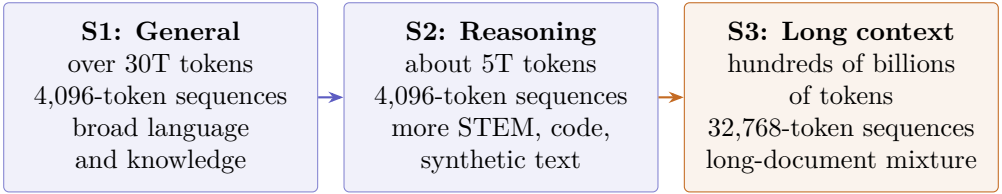
Quality, diversity, and domain proportions affect what the model learns from a fixed token budget.

Qwen3 combines real and synthetic data and tunes which examples are seen—not just how many are collected.

Source: Qwen Team (2025), Section 3.1; conceptual schematic.

Qwen3's Pre-training Pipeline

One objective, three stages of training.



What changes: the **data mixture**, the **sequence length**, and parts of the optimization schedule.

A curriculum can change the model's capabilities while it is still trained with a causal prediction loss.

Source: Qwen Team (2025), Section 3.2; token counts are approximate

Reasoning Stage: Shifting the Data Mixture

Why shift toward STEM, code, and reasoning-oriented text late in training?

After S1

Broad language and world knowledge from over 30T tokens.

During S2

Continue training on about 5T higher-quality tokens, with greater emphasis on STEM, coding, reasoning, and synthetic data.

What changes in S2?

- **Data mixture:** More reasoning-intensive content.
- **Learning schedule:** Faster learning-rate decay.
- **Objective:** Still next-token prediction.

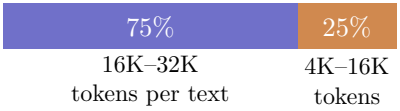
S2 strengthens reasoning-related capabilities by changing the training data and learning schedule, not the prediction objective.

Source: Qwen Team (2025), Section 3.2

Long-context Pre-training

Why learn on 4K-token sequences before training at 32K?

S3 changes the length distribution



The reported long-context corpus favors long documents.

Progressive context extension

$$4\text{K} \xrightarrow{\text{S3 training}} 32\text{K}$$

$$32\text{K} \xrightarrow{\text{inference methods}} 128\text{K}$$

Longer attention is costlier; the report also adjusts RoPE and uses YaRN/DCA for inference extension.

Start with short-context learning, then expose the model to long-range dependencies. A 128K supported window is not 128K of S3 training.

Source: Qwen Team (2025), Sections 2 and 3.2; 128K applies to larger variants

The Result: Qwen3 Base

Evaluate what pre-training has produced—before instruction tuning.

	Qwen2.5-72B Base	Qwen2.5-Plus Base	Llama-4-Maverick Base	DeepSeek-V3 Base	Qwen3-235B-A22B Base
Architecture	Dense	MoE	MoE	MoE	MoE
# Total Params	72B	271B	402B	671B	235B
# Activated Params	72B	37B	17B	37B	22B
General Tasks					
MMLU	86.06	85.02	85.16	87.19	**87.81**
MMLU-Redux	83.91	82.69	84.05	86.14	**87.40**
MMLU-Pro	58.07	63.52	63.91	59.84	**68.18**
SuperGPQA	36.20	37.18	40.85	41.53	**44.06**
BBH	86.30	85.60	83.62	86.22	**88.87**

Bigger is not always better: Qwen3 achieves stronger base-model performance than several larger models, highlighting the importance of architecture and pre-training choices beyond parameter count.

Source: Qwen Team (2025), Section 3.3 and Table 3

PART 6 / 06

From Pre-training to Post-training

Does strong next-token prediction guarantee a helpful assistant?

We'll cover: the base checkpoint and the next learning signals

Is a Base Model an Assistant?

Predicting likely text is not the same objective as following a user's instruction.

Base model: plausible continuation A prompt may be continued as a document, a Q&A dataset, or a conversation transcript.

Illustrative pattern:

```
Q: Explain RoPE in two bullets.
A: ...
Q: ...
```

Assistant: instruction-oriented Post-training encourages following the request, format, and intended role.

Illustrative pattern:

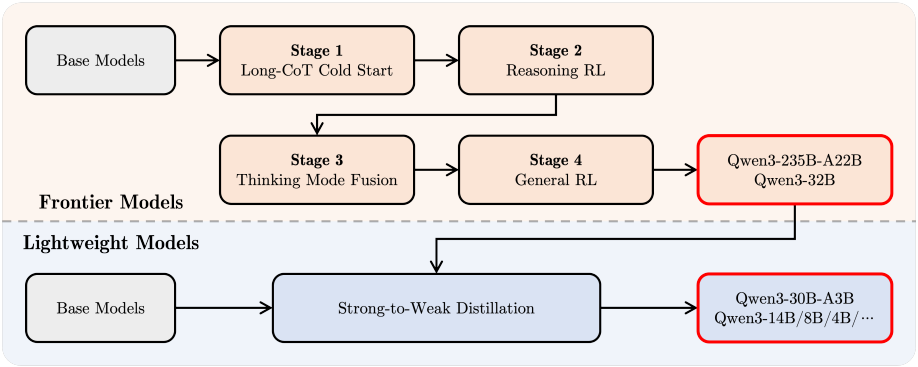
```
1. RoPE rotates position-
dependent query/key pairs.
2. It encodes relative position.
```

A base model can sometimes follow instructions, but is not tuned for consistent assistant behavior.

Pre-training builds a broadly useful language model. Assistant behavior requires additional training signals.

Source: Qwen Team (2025), Sections 3–4

What's Next? Post-training



Pre-training learns to model text; post-training steers that ability toward useful interaction.

Source: Qwen Team (2025), Sections 3–4; diagram is a simplified teaching view

References: objectives, transfer, and scaling

- Devlin et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.*
  NAACL 2019
- Raffel et al. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.*
  JMLR 21(140)
- Kaplan et al. (2020). *Scaling Laws for Neural Language Models.*
  arXiv:2001.08361
- Hoffmann et al. (2022). *Training Compute-Optimal Large Language Models.*
  arXiv:2203.15556
- Stanford CS224N. Pre-training lecture; explanatory figures adapted with attribution.
  Course materials

References: modern recipe and multimodal extension

- Qwen Team (2025). *Qwen3 Technical Report.*
  arXiv:2505.09388v1; §§2–3, Tables 1–3 for the case study; §4 for the post-training boundary.
- Meta (2024). *Introducing Meta Llama 3.*
  Official release article; “Scaling up pretraining” for the 200B versus 15T comparison.
- Yu et al. (2023). *Scaling Autoregressive Multi-Modal Models: Pretraining and Instruction Tuning.*
  arXiv:2309.02591v1; §2 and Figure 8 for the multimodal token sequence.

Paper figures and reported scores retain source labels. Explanatory schematics and both exercises are constructed for this seminar.